

MATERIAL DE ESTUDO

Fundamentos de Machine Learning

Large Data Models e Transformers

NeoSpace — Preparação para entrevista

Do pipeline clássico ao LDM — conceitos, matemática e prática

Use este material para escrever no caderno e fixar os conceitos.

Módulo 1 — O Problema que o LDM Resolve

1.1 O pipeline clássico de ML

Antes de entender o LDM, é preciso entender o que ele substitui. O pipeline tradicional de Machine Learning em empresas segue sempre a mesma estrutura:

Pipeline clássico — 5 etapas

1. Coleta de dados brutos (logs, transações, eventos)
2. Feature engineering — humanos decidem quais variáveis criar
3. Feature store — banco de dados de features calculadas
4. Modelos especializados — um modelo por tarefa (fraude, churn, crédito...)
5. Camada de regras de negócio — decisões codificadas manualmente

Cada etapa tem um custo de manutenção. O problema central: quando o mercado muda (ex: chegada do Pix), todas as features precisam ser redesenhadas do zero, o que pode levar meses.

1.2 O Feature Engineering Wall

O whitepaper da NeoSpace identifica 10 limitações estruturais do pipeline clássico. As mais importantes para memorizar:

Rigidez temporal	Mudanças de mercado invalidam features. O modelo degrada e demora meses para ser corrigido.
Mean tyranny	Médias apagam picos comportamentais críticos. 3 reclamações em 2 dias ≠ 3 reclamações em 60 dias — a média é igual.
Knowledge imprinting	Bias humano embutido nas features. O modelo confirma hipóteses existentes em vez de descobrir padrões novos.
Explosão de modelos	Uma empresa pode ter 80+ modelos em produção, cada um com feature engineering separado e equipe dedicada.
Inexpressividade de texto	Feature stores não conseguem processar texto (chats, tickets, notas) de forma significativa.

1.3 A solução: Large Data Model (LDM)

O LDM é um transformer treinado diretamente no event stream bruto — a sequência completa de eventos do cliente — sem feature engineering manual. A analogia com AlphaZero é central:

Pipeline clássico	LDM
Feature engineering manual	Event stream bruto — sem features
Um modelo por tarefa	Um modelo unificado — múltiplas tarefas
Features agregadas (médias)	Sequência completa de eventos
Rígido a mudanças de mercado	Aprende mudanças estruturalmente
Não processa texto	Texto é um campo nativo
Custo: N equipes × N modelos	Custo: 1 modelo + fine-tuning

Módulo 2 — Fundamentos Matemáticos

2.1 O Score — como o modelo toma decisões

Todo modelo de classificação produz um score: um número entre 0 e 1 que representa a probabilidade estimada de pertencer à classe positiva (ex: fraude, churn).

Soma ponderada (z)

O primeiro passo é calcular a soma ponderada das features com os pesos aprendidos no treinamento:

$$z = w_1 \times x_1 + w_2 \times x_2 + \dots + w_n \times x_n + b$$

w = pesos | x = features | b = bias

O que é o bias (b)?

O bias é o 'ponto de partida' do modelo. Sem ele, a fronteira de decisão seria forçada a passar pela origem, limitando o que o modelo pode aprender. Com o bias, o modelo tem mais liberdade para se ajustar aos dados.

Função Sigmoid

O z pode ser qualquer número (positivo, negativo, grande, pequeno). A sigmoide 'espreme' esse valor para ficar entre 0 e 1, tornando-o interpretável como probabilidade:

$$\text{score} = 1 / (1 + e^{-z})$$

e = número de Euler ≈ 2.718

z	Cálculo	Score	Interpretação
-3	1/(1+e³)	0.05	5% — praticamente negativo
-1	1/(1+e¹)	0.27	27% — provavelmente negativo
0	1/(1+e⁰)	0.50	50% — sem certeza
1	1/(1+e⁻¹)	0.73	73% — provavelmente positivo

3	$1/(1+e^{-3})$	0.95	95% — praticamente positivo
---	----------------	------	-----------------------------

2.2 Os 4 Quadrantes — Matriz de Confusão

Após o modelo fazer previsões, comparamos com a realidade. Existem 4 resultados possíveis:

	Modelo disse: POSITIVO	Modelo disse: NEGATIVO
Era: POSITIVO	TP — True Positive Acertou ✓ Ex: disse fraude, era fraude	FN — False Negative Fraude passou ✗ Custo: prejuízo financeiro
Era: NEGATIVO	FP — False Positive Alarme falso ✗ Custo: cliente bloqueado	TN — True Negative Acertou ✓ Ex: disse legítima, era legítima

2.3 Métricas de Avaliação

Precision e Recall

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Quando acuso, estou certo?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

De todas as fraudes reais, quantas encontrei?

A tensão fundamental

Aumentar Recall (encontrar mais fraudes) → reduz Precision (mais alarmes falsos)

Aumentar Precision (menos alarmes) → reduz Recall (mais fraudes passam)

O negócio decide qual erro é mais caro. Não existe configuração perfeita.

AUROC e AUPRC

Ambas medem a qualidade do modelo como um todo, independente do limiar escolhido:

AUROC

De cada par (positivo, negativo), em quantos o modelo coloca o positivo acima?
 0.5 = aleatório, 1.0 = perfeito. Bom para datasets balanceados.

AUPRC	Equilíbrio entre Precision e Recall ao longo de todos os limiares. Melhor para datasets desbalanceados (fraude tem 0.17% de casos positivos).
F1 Score	Média harmônica entre Precision e Recall. $F1 = 2 \times (\text{Prec} \times \text{Rec}) / (\text{Prec} + \text{Rec})$. Bom para uma única métrica de equilíbrio.

2.4 Overfitting e Underfitting

Underfitting	Bom Modelo	Overfitting
Estudou pouco. Erra em tudo — no treino e no teste.	Aprendeu os padrões reais. Vai bem no treino e no teste.	Decorou as respostas. Perfeito no treino, péssimo no teste.
AUROC treino: 0.61 AUROC teste: 0.59	AUROC treino: 0.87 AUROC teste: 0.84	AUROC treino: 0.99 AUROC teste: 0.71

Regra prática: se $(\text{AUROC treino} - \text{AUROC teste}) > 0.05 \rightarrow \text{overfitting}$. O patience no TabNet é um mecanismo anti-overfitting: para quando o desempenho no teste para de melhorar.

Módulo 3 — Arquitetura Transformer

3.1 O que é um Transformer

Transformer é uma arquitetura de rede neural criada em 2017 (paper 'Attention is All You Need', Vaswani et al.). Revolucionou NLP e agora é a base dos LDMs para dados comportamentais.

A intuição central

Um transformer lê uma sequência de tokens e aprende quais tokens são mais relevantes para entender cada outro token. Isso é o mecanismo de atenção (self-attention).

No LDM: cada evento do cliente é um token. O transformer aprende que a combinação 'troca de senha → compra negada → compra internacional' é muito mais relevante para avaliar a transação atual do que um login normal de 30 dias atrás.

3.2 Conceitos-chave do Transformer

Token	Unidade mínima de entrada. Em texto: uma palavra. No LDM: um evento do cliente (uma transação, uma reclamação, um login).
Embedding	Vetor de N dimensões que representa um token. Não é normalização — é uma representação geométrica onde tokens similares ficam próximos no espaço. 'São Paulo' e 'Rio de Janeiro' ficam próximos; 'São Paulo' e 'Tóquio' ficam distantes.
Positional Encoding	Como o transformer não tem noção de ordem, somamos ao embedding um vetor que codifica a posição do token na sequência. Sem isso, o transformer trataria a sequência como um saco desordenado.
Self-Attention	Mecanismo que calcula, para cada token, o quanto cada outro token da sequência é relevante. Produz pesos de atenção: o evento 'troca de senha' pode receber peso 0.38, enquanto 'login normal' recebe 0.03.
Multi-head Attention	Múltiplas 'cabeças' de atenção em paralelo, cada uma aprendendo um aspecto diferente. Uma cabeça pode capturar padrões temporais, outra padrões de valor, outra padrões de localização.

Época (Epoch)	Uma passagem completa pelo dataset de treinamento. O modelo ajusta os pesos ao final de cada época. Mais épocas = mais aprendizado, mas risco de overfitting.
Batch	Subconjunto dos dados processado de uma vez. Batch size 512 = o modelo processa 512 exemplos antes de atualizar os pesos. Batch maior = treinamento mais rápido, mais memória.
Patience	Critério de parada antecipada. Patience=10 significa: se depois de 10 épocas sem melhora no conjunto de validação, para o treinamento. Evita overfitting.
Learning Rate	Taxa de aprendizado — o tamanho do passo ao ajustar os pesos. Muito alto = aprendizado instável. Muito baixo = aprendizado lento.

3.3 O Mecanismo de Atenção — Q, K, V

O mecanismo de atenção usa três vetores derivados de cada token:

Vetor	Nome	Função
Q	Query (Pergunta)	O que o token atual está procurando? 'Quais eventos passados me ajudam a entender?'
K	Key (Chave)	O que cada token passado oferece? É o 'título' que cada evento expõe.
V	Value (Valor)	O conteúdo real do evento, usado quando o Query combina com o Key.

$$\text{Atenção}(Q, K, V) = \text{softmax} \left(\frac{Q \times K^T}{\sqrt{d}} \right) \times V$$

d = dimensão dos vetores (normalização para estabilidade)

Analogia: busca no Google

Q = o que você digita na busca

K = os títulos dos resultados

V = o conteúdo das páginas

A atenção = o score de relevância entre sua busca e cada título

Módulo 4 — Arquitetura do LDM

4.1 Os 5 Blocos da Arquitetura

O LDM da NeoSpace tem 5 blocos distintos, cada um com um transformer especializado:

#	Bloco	O que faz
1	Field Encoders	Converte cada campo (número, categoria, texto, timestamp) em um vetor de embedding. 'Tóquio' vira um ponto no espaço vetorial, não apenas '1 = localização nova'.
2	Local Fusion	Combina os embeddings dos campos de um mesmo evento. 'Valor alto' + 'localização nova' + 'horário suspeito' juntos criam um sinal diferente de cada um isolado.
3	Temporal Backbone	O transformer principal. Lê a sequência completa de eventos e aplica self-attention para aprender quais eventos passados são relevantes para o atual.
4	Spatial Backbone	Aplica atenção entre clientes diferentes. Aprende que este cliente compartilha padrões com outros que sofreram fraude — mesmo sem histórico suficiente.
5	Latent-State Processor	Converte o vetor latente final (que acumula todo o contexto) em uma decisão ou score. A sigmoide final é a mesma do modelo clássico — o que muda é o vetor que chega nela.

4.2 O Ciclo de Valor em 3 Fases

Fase 1 Pre-train	Fase 2 Predictive	Fase 3 Prescriptive
O modelo lê todo o histórico da empresa sem labels. Aprende o que é normal, quais sequências precedem quais eventos.	Fine-tune para tarefas específicas: 'esse cliente vai churnar?' O conhecimento do pré-treino acelera e melhora o aprendizado.	O modelo aprende a tomar decisões otimizadas respeitando restrições. Absorve a camada de regras de negócio — o maior salto de valor.

4.3 Clássico vs LDM — O que cada um vê

Exemplo concreto: detecção de fraude Transação: R\$4.200 em Tóquio, às 4h22, cartão nunca usado no Japão
--

Modelo clássico — vê 1 transação isolada:

features: valor_norm=0.84, local_novo=1, horario_suspeito=1

score: 0.44 → abaixo do limiar 0.5 → FRAUDE PASSA

LDM — vê a sequência de 8 eventos:

Evento 6: troca de senha em IP estrangeiro às 3h14 (atenção: 0.38)

Evento 7: compra de R\$3.800 negada por limite (atenção: 0.35)

Evento 8: compra Tóquio R\$4.200 (evento atual)

score: 0.94 → acima do limiar → FRAUDE BLOQUEADA

Módulo 5 — Glossário Completo de Termos

Todos os termos importantes do curso, em ordem alfabética:

AUROC	Area Under the ROC Curve. Mede o poder de separação do modelo entre classes. 0.5=aleatório, 1.0=perfeito. Bom para datasets balanceados.
AUPRC	Area Under the Precision-Recall Curve. Melhor para datasets desbalanceados (fraude, churn). Mede o equilíbrio entre Precision e Recall.
Batch	Subconjunto de dados processado de uma vez antes de atualizar os pesos. Batch maior = mais rápido, mais memória.
Bias (b)	Parâmetro extra na soma ponderada. Permite ao modelo ajustar a fronteira de decisão sem depender da origem.
Churn	Cancelamento de serviço por um cliente. Taxa de churn = % de clientes que cancelaram em um período.
Embedding	Vetor de N dimensões que representa um item (palavra, evento, localização). Tokens similares ficam próximos no espaço vetorial.
Época (Epoch)	Uma passagem completa pelo dataset de treinamento. Mais épocas = mais aprendizado, com risco de overfitting.
Event Stream	Sequência de eventos de um cliente ordenada por tempo. Base do LDM — em vez de features agregadas, o modelo recebe a sequência bruta.
F1 Score	Média harmônica entre Precision e Recall. $F1 = 2 \times (P \times R) / (P + R)$. Única métrica para avaliar equilíbrio.
False Negative (FN)	Modelo disse negativo, era positivo. Fraude que passou, cherner não identificado. Custo direto.
False Positive (FP)	Modelo disse positivo, era negativo. Alarme falso. Cliente legítimo bloqueado. Custo indireto.

Feature Engineering	Processo manual de criar variáveis (features) a partir de dados brutos. O 'gargalo' que o LDM elimina.
Feature Store	Banco de dados de features pré-calculadas. Facilita o reuso mas introduz rigidez e custo de manutenção.
Fine-tuning	Ajuste fino de um modelo pré-treinado para uma tarefa específica. Mais rápido e preciso que treinar do zero.
LDM	Large Data Model. Transformer treinado diretamente em event streams brutos. Unifica múltiplos modelos especializados.
Learning Rate	Taxa de ajuste dos pesos a cada iteração. Muito alto = instável. Muito baixo = lento.
Limiar (Threshold)	Valor de corte aplicado ao score para tomar a decisão. $\text{Score} \geq \text{limiar} \rightarrow \text{positivo}$. Ajustável conforme o custo de cada erro.
Overfitting	Modelo decorou o dataset de treino. Perfeito no treino, ruim no teste. Detectado por: $\text{AUROC}_{\text{treino}} \gg \text{AUROC}_{\text{teste}}$.
Patience	Critério de parada antecipada. Para o treinamento se N épocas passarem sem melhora no conjunto de validação.
Positional Encoding	Vetor somado ao embedding para indicar a posição do token na sequência. Dá ao transformer noção de ordem temporal.
Pre-training	Treinamento inicial sem labels. O modelo aprende representações gerais antes de ser especializado.
Precision	$\text{TP} / (\text{TP} + \text{FP})$. Dos que o modelo acusou como positivos, quantos eram de verdade? 'Quando acuso, estou certo?'
Recall	$\text{TP} / (\text{TP} + \text{FN})$. De todos os positivos reais, quantos o modelo encontrou? 'Encontrei todas as fraudes?'

Score	Número entre 0 e 1 produzido pelo modelo para cada exemplo. Representa a probabilidade estimada de pertencer à classe positiva.
Self-Attention	Mecanismo central do transformer. Calcula o quanto cada token deve prestar atenção em cada outro token da sequência.
Sigmoide	Função que transforma z (qualquer número) em score entre 0 e 1. $\text{score} = 1/(1+e^{-z})$.
TabNet	Transformer para dados tabulares. Usa atenção sobre features. Proxy do LDM para datasets estáticos.
Token	Unidade mínima de entrada do transformer. Em NLP: palavra. No LDM: evento do cliente.
Transformer	Arquitetura de rede neural baseada em atenção (Vaswani et al., 2017). Base dos LLMs e LDMs.
True Negative (TN)	Modelo disse negativo, era negativo. Acertou. Transação legítima classificada como legítima.
True Positive (TP)	Modelo disse positivo, era positivo. Acertou. Fraude identificada como fraude.
Underfitting	Modelo não aprendeu nem o básico. Ruim tanto no treino quanto no teste. Modelo muito simples ou dados insuficientes.
Vetor (embedding)	Representação numérica de um objeto em espaço multidimensional. Cada dimensão captura um conceito aprendido.
XGBoost	Gradient Boosting extremo. Modelo de árvores de decisão em ensemble. Domina dados tabulares estáticos.
z (soma ponderada)	Input da sigmoide. $z = w_1x_1 + w_2x_2 + \dots + b$. Pode ser qualquer número real.

